

PhishGuard AI: Technology Stack & Tools

This document outlines every major tool, framework, and library used to build the PhishGuard AI project, along with the technical justification for *why* it was chosen over alternatives. This is excellent for defending your architectural choices to the panel.

1. Cloud & Infrastructure

- **Render.com (Cloud Hosting)**
 - **Why:** Chosen for deploying the live backend server. Render provides seamless integration with Python/Flask, automatic scaling, and secure HTTPS endpoints out of the box, ensuring the browser extension can securely reach the “PhishGuard AI” API from anywhere.
- **Gunicorn (WSGI HTTP Server)**
 - **Why:** Flask’s built-in server is not meant for production. Gunicorn was used (via the `Procfile`) because it handles multiple concurrent requests efficiently, ensuring that if multiple users browse websites simultaneously, the server won’t crash.

2. Backend & API

- **Python 3.x**
 - **Why:** The industry standard for Machine Learning. It allowed us to seamlessly integrate our ML models, data processing pipelines, and web server logic in a single unified language.
- **Flask (Web Framework)**
 - **Why:** We chose Flask over Django because it is lightweight and unopinionated. We needed a fast, low-latency API to receive JSON payloads from the extension and return ML predictions. Flask provided exactly this without unnecessary bloat.

3. Frontend (Browser Extension)

- **Vanilla JavaScript, HTML, & CSS**
 - **Why:** Used to build the extension and the Simulation Lab UI. Vanilla JS was chosen to keep the extension extremely lightweight, ensuring it doesn’t consume excessive RAM or slow down the user’s browser.
- **Chrome Extensions API (Manifest V3)**
 - **Why:** Used to securely traverse the Document Object Model (DOM). This API allows the script (`background.js`) to silently run in the background, intercepting the URL and scraping text/images asynchronously via `fetch()` without interrupting the user’s workflow.

4. Core Machine Learning & Math

- **Scikit-Learn (`sklearn`)**
 - **Why:** Used extensively in the URL pipeline. We used it to build our `VotingClassifier` ensemble (Random Forest, Gradient Boosting, Logistic Regression). Tree-based models from `sklearn` mathematically outperform deep learning models on tabular data (like URL string features) and train much faster.
- **SciPy**
 - **Why:** Used in the Core Fusion Engine (`phishfusion.py`). Specifically, we used `scipy.spatial.distance.cosine` to calculate the Cosine Similarity between our 768-dimensional NLP vectors and 64-dimensional Vision vectors to mathematically prove if the models agreed on a threat.
- **NumPy & Pandas**
 - **Why:** Used for high-performance array operations and dataset manipulation. Pandas allowed us to easily manage and clean the hundreds of thousands of rows of URL data during the training phase.
- **Joblib**
 - **Why:** Used to serialize (save) and deserialize (load) the trained ML models (`.pkl` files). Joblib is highly optimized for Python objects that carry large NumPy arrays (like our Random Forest trees), making inference significantly faster than using standard `pickle`.

5. Modality-Specific Data Processing

- **tlextract (URL Parsing)**
 - **Why:** Used in `phishurl.py`. Standard URL parsers fail on complex country-code domains (like `.co.in` or `.gov.in`). `tlextract` uses a live public suffix list to accurately separate the Top Level Domain (TLD), allowing us to extract precise structural features without errors.
- **Natural Language Processing (NLP Frameworks)**
 - **Why:** Used in `phishtext.py` to generate the 768-dimensional embeddings. Standard keyword matching is easily bypassed by attackers. NLP algorithms understand the contextual semantics of urgency and deception.
- **Computer Vision (CNNs) & Optical Character Recognition (OCR)**
 - **Why:** Used in `phishimage.py`. Attackers embed text inside images to bypass text-scrappers. OCR extracts this text, and CNNs generate the 64-dimensional visual embeddings required to detect slight alterations in spoofed brand logos.
- **Audio/Video Processing Libraries**
 - **Why:** Used in `phishaudio.py` and `phishvideo.py` to slice video into discrete frames and extract audio waveforms. Essential for de-

tecting the digital artifacts left behind by AI-generated deepfake media.